6 מימוש פרויקט הגמר

6.1 מבוא: מהתיאוריה למעשה

לאורך הספר עברנו מסע בעולם סוכני הבינה המלאכותית וליגת Q21G. כעת הגיע הזמן לרכז את כל הידע לכדי מימוש מעשי. פרק זה מספק הנחיות מפורטות למימוש סוכן שחקן וסוכן שופט.

6.1.1 מטרות הפרק

בסיום פרק זה, תבינו:

- את הגדרת סביבת הפיתוח הנדרשת
- את מבנה סוכן השחקן ואחריותו
- את מבנה סוכן השופט ואחריותו
- את קבצי התצורה הנדרשים
- את אסטרטגיות הבדיקה המומלצות

6.2 סביבת הפיתוח

6.2.1 דרישות מערכת

טבלה 59: דרישות סביבה

רכיב	דרישה
שפת תכנות	Python מומלץ, אך כל שפה אחרת מותרת
Gmail API	2 חשבונות Gmail עם הרשאות OAuth 2.0 (ראו נספח א' ונספח ה')
CLI Terminal	Anthropic או Gemini מומלץ, אך כל LLM CLI אחר אפשרי
Git	לניהול גרסאות
PostgreSQL	הקמת 2 מסדי נתונים, אחד לכל סוכן AI

6.2.2 התקנת ספריות

התקנת ספריות נדרשות

```
# requirements.txt
google-api-python-client>=2.100.0
google-auth-httplib2>=0.1.0
google-auth-oauthlib>=1.0.0
anthropic>=0.20.0  # or openai>=1.0.0
python-dotenv>=1.0.0
aiohttp>=3.9.0
psycopg2-binary>=2.9.0  # optional, for PostgreSQL
```

6.2.3 מבנה הפרויקט המומלץ

מבנה ספריות הפרויקט

```
q21g-agents/├──
 src/│
   ├── player/│
   │   ├── __init__.py│
   │   ├── player_agent.py      # Main player logic│
   │   ├── question_generator.py # Question generation│
   │   └── guess_strategy.py    # Guessing strategy│
   ├── referee/│
   │   ├── __init__.py│
   │   ├── referee_agent.py     # Main referee logic│
   │   ├── paragraph_selector.py # Paragraph selection│
   │   └── scorer.py            # Scoring logic│
   ├── shared/│
   │   ├── __init__.py│
   │   ├── gmail_client.py      # Gmail API wrapper│
   │   ├── protocol.py          # Message types│
   │   └── config.py            # Configuration│
   └── main.py                  # Entry point├──
 config/│
   ├── credentials.json         # Gmail OAuth credentials│
   ├── player.env               # Player configuration│
   └── referee.env              # Referee configuration├──
 data/│
   └── course_materials/        # Course materials PDFs├──
 tests/│
   ├── test_player.py│
   ├── test_referee.py│
   └── test_protocol.py├──
```

```
requirements.txt└──
README.md
```

6.3 מימוש סוכן השחקן

6.3.1 מחזור החיים של השחקן

1. **אתחול** — טעינת תצורה וחיבור ל-Gmail API
2. **רישום** — שליחת LEAGUE_REGISTER_REQUEST
3. **המתנה** — מעקב אחר הודעות נכנסות
4. **משחק** — תגובה להזמנות וניהול שלבי המשחק
5. **סיום** — סגירת חיבורים ושמירת לוגים

6.3.2 רצף הודעות שחקן מלא

טבלה 60: הודעות שחקן - רישום ועונה

כיוון	סוג הודעה	תיאור
שליחה	LEAGUE_REGISTER_REQUEST	רישום לליגה
קבלה	RESPONSE_LEAGUE_REGISTER	אישור רישום + טבלת משתמשים
קבלה	BROADCAST_START_SEASON	התחלת עונה
שליחה	SEASON_REGISTRATION_REQUEST	רישום לעונה
קבלה	RESPONSE_SEASON_REGISTRATION	אישור רישום לעונה
קבלה	BROADCAST_ASSIGNMENT_TABLE	טבלת הקצאות

כל הזכויות שמורות - Dr. Segal Yoram ©

טבלה 61: הודעות שחקן - משחק

כיוון	סוג הודעה	תיאור
קבלה	GAME_INVITATION	הזמנה למשחק
שליחה	GAME_JOIN_ACK	אישור הצטרפות (ידני, לא אוטומטי)
קבלה	Q21_WARMUP_CALL	שאלת חימום
שליחה	Q21_WARMUP_RESPONSE	תשובה לחימום
קבלה	Q21_ROUND_START	התחלת סיבוב + מידע על ספר
קבלה	Q21_QUESTIONS_CALL	בקשת שאלות
שליחה	Q21_QUESTIONS_BATCH	20 שאלות (תוך 300s)
קבלה	Q21_ANSWERS_BATCH	תשובות לשאלות
שליחה	Q21_GUESS_SUBMISSION	ניחוש סופי
קבלה	Q21_HINTS	רמזים נוספים
קבלה	Q21_GUESS_RESULT	תוצאות הניחוש
קבלה	Q21_SCORE_FEEDBACK	משוב ציון
קבלה	GAME_OVER	סיום משחק

6.3.3 מודול יצירת שאלות

דוגמה ליצירת שאלות

```
from anthropic import Anthropic

class QuestionGenerator:
    """Generates 20 multiple-choice questions based on hint."""

    def __init__(self, model: str = "claude-sonnet-4-20250514"):
        self.client = Anthropic()
        self.model = model

    async def generate_questions(
        self,
        description: str,
        association_topic: str
    ) -> list[dict]:
        """Generate 20 questions based on hint."""
        prompt = f"""
        Based on this description: "{description}"
        And association topic: "{association_topic}"

        Generate 20 strategic multiple-choice questions
        to identify the source paragraph.
```

```
        Each question should have 4 options.
        """

        response = await self.client.messages.create(
            model=self.model,
            max_tokens=4096,
            messages=[{"role": "user", "content": prompt}]
        )

        return self._parse_questions(response.content[0].text)
```

6.3.4 אסטרטגיית הניחוש

טיפ למימוש

טיפים לאסטרטגיית ניחוש:

- נתחו את התשובות לזיהוי דפוסים
- חפשו בחומרי הקורס פסקאות תואמות
- שקללו את הביטחון בכל ניחוש
- זכרו: המילה האסוציאטיבית שווה 30% מהציון!

6.4 מימוש סוכן השופט

6.4.1 מחזור החיים של השופט

1. **אתחול** — טעינת חומרי הקורס
2. **רישום** — שליחת REFEREE_REGISTER_REQUEST
3. **הקצאה** — קבלת טבלת הקצאות
4. **ניהול משחק** — הזמנה, רמזים, תשובות, ציונים
5. **דיווח** — שליחת GAME_RESULT

6.4.2 רצף הודעות שופט מלא

טבלה 62: הודעות שופט - רישום

כיוון	סוג הודעה	תיאור
שליחה	REFEREE_REGISTER_REQUEST	רישום כשופט
קבלה	RESPONSE_REFEREE_REGISTER	אישור רישום
קבלה	BROADCAST_ASSIGNMENT_TABLE	משחקים שהוקצו

כל הזכויות שמורות - Dr. Segal Yoram ©

טבלה 63: הודעות שופט - ניהול משחק

כיוון	סוג הודעה	תיאור
שליחה	Q21_WARMUP_CALL	שליחת שאלת חימום
קבלה	Q21_WARMUP_RESPONSE	תשובת חימום מהשחקן
שליחה	Q21_ROUND_START	תחילת סיבוב (כולל מידע על הספר)
קבלה	Q21_QUESTIONS_BATCH	20 שאלות מהשחקן
שליחה	Q21_ANSWERS_BATCH	תשובות + deadline לניחוש
קבלה	Q21_GUESS_SUBMISSION	ניחוש סופי מהשחקן
שליחה	Q21_SCORE_FEEDBACK	תוצאות לשחקן
שליחה	MATCH_RESULT_REPORT	דיווח למנהל הליגה

הבדלים מהמפרט המקורי

הקוד בפועל שונה מהמפרט:

- GAME_JOIN_ACK/GAME_INVITATION — מוגדרים אך לא בשימוש (חימום מתחיל ישירות)
- Q21_QUESTIONS_CALL — מוזג לתוך Q21_ROUND_START
- GAME_OVER — הוחלף ב-Q21_SCORE_FEEDBACK
- GAME_RESULT — שונה ל-MATCH_RESULT_REPORT

6.4.3 בחירת פסקה ויצירת רמזים

בחירת פסקה

```
import random

class ParagraphSelector:
    """Selects paragraph and creates hints."""

    def __init__(self, materials_path: str):
        self.materials = self._load_materials(materials_path)

    def select_paragraph(self) -> dict:
        """Select a random paragraph from course materials."""
        lecture = random.choice(self.materials)
        paragraph = random.choice(lecture["paragraphs"])

        return {
            "lecture_id": lecture["id"],
            "paragraph_id": paragraph["id"],
            "text": paragraph["text"],
```

```
            "opening_sentence": paragraph["text"].split(".")[0] + "."
        }

    def create_hints(self, paragraph: dict) -> dict:
        """Create description and association for paragraph."""
        # Use LLM to generate hints
        # IMPORTANT: description must NOT contain words from paragraph!
        return {
            "secret_title": "...",  # Max 5 words
            "description": "...",    # Max 15 words, no overlap
            "association_word": "...",
            "association_topic": "..."
        }
```

6.4.4 חישוב ציונים

חישוב ציון סיבוב

```
class Scorer:
    """Scores player guesses against original paragraph."""

    def score_guess(
        self,
        original: dict,
        guess: dict
    ) -> dict:
        """Calculate score (0-100) with breakdown."""

        # 50% - Opening sentence accuracy
        sentence_score = self._compare_sentences(
            original["opening_sentence"],
            guess["opening_sentence"]
        )

        # 20% - Sentence reasoning quality
        reasoning_score = self._evaluate_reasoning(
            guess["sentence_reasoning"]
        )

        # 20% - Association word accuracy
        association_score = self._compare_association(
            original["association_word"],
            guess["association_word"],
            guess["association_variations"]
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
        )

        # 10% - Association reasoning quality
        assoc_reasoning_score = self._evaluate_reasoning(
            guess["association_reasoning"]
        )

        total = (
            sentence_score * 0.5 +
            reasoning_score * 0.2 +
            association_score * 0.2 +
            assoc_reasoning_score * 0.1
        )

        return {
            "total": round(total, 2),
            "breakdown": {
                "opening_sentence": sentence_score,
                "sentence_reasoning": reasoning_score,
                "association_word": association_score,
                "association_reasoning": assoc_reasoning_score
            }
        }
```

6.4.5 המרת ציון פרטי לציון ליגה

לאחר חישוב הציון הפרטי (0–100), השופט ממיר אותו לציון ליגה (0–3):

טבלה 64: המרת ציון פרטי לציון ליגה

טווח ציון פרטי	ציון ליגה	רמת ביצוע
100–85	3 נקודות	מצוין
84–70	2 נקודות	טוב
69–50	1 נקודה	בינוני
49–0	0 נקודות	חלש

ScoreCalculator — Full Implementation

```
class ScoreCalculator:
    """Complete scoring implementation for Q21 referee."""

    WEIGHTS = {
        "opening_sentence": 0.50,
```

```
        "sentence_justification": 0.20,
        "associative_word": 0.20,
        "word_justification": 0.10
    }

    def calculate_scores(self, original: dict, guess: dict) -> dict:
        """Calculate complete scores with feedback."""
        # Calculate component scores (0-100 each)
        components = {
            "opening_sentence_score": self._score_sentence(
                original["opening_sentence"], guess["opening_sentence"])
,
            "sentence_justification_score": self._score_justification(
                guess["sentence_reasoning"]),
            "associative_word_score": self._score_word(
                original["association_word"], guess["association_word"])
,
            "word_justification_score": self._score_justification(
                guess["association_reasoning"])
        }

        # Calculate weighted private score (0-100)
        private_score = sum(
            components[f"{k}_score"] * v
            for k, v in self.WEIGHTS.items()
        )

        # Convert to league score (0-3)
        league_score = self._convert_to_league_score(private_score)

        # Generate 30-50 word feedback for each component
        feedback = self._generate_feedback(original, guess, components)

        return {
            **components,
            "private_score": round(private_score, 2),
            "league_score": league_score,
            "opening_sentence_feedback": feedback["sentence"],
            "word_feedback": feedback["word"]
        }

    def _convert_to_league_score(self, private_score: float) -> int:
        """Convert private score (0-100) to league score (0-3)."""
        if private_score >= 85:
            return 3
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
        elif private_score >= 70:
            return 2
        elif private_score >= 50:
            return 1
        return 0

    def _generate_feedback(self, original: dict, guess: dict,
                           components: dict) -> dict:
        """Generate 30-50 word feedback for each component."""
        # Use LLM to generate meaningful feedback
        return {
            "sentence": self._generate_component_feedback(
                "opening_sentence", original, guess, components),
            "word": self._generate_component_feedback(
                "associative_word", original, guess, components)
        }
```

חובת הסבר מילולי

השופט **חייב** לספק הסבר מילולי באורך 30–50 מילים לכל רכיב ציון. ההסבר צריך לנמק את הציון שניתן ולהתייחס לאיכות הניחוש ביחס לתשובה הנכונה.

6.5 קבצי תצורה

6.5.1 תצורת השחקן

vne.reyalp

```
# Player Configuration
PLAYER_EMAIL=MyGroup.player@gmail.com
PLAYER_NAME=MyGroup Player
LEAGUE_MANAGER_EMAIL=bitalevi100@gmail.com
LOG_SERVER_EMAIL=beit.halevi.700@gmail.com

# LLM Configuration
LLM_PROVIDER=anthropic
LLM_MODEL=claude-sonnet-4-20250514
LLM_API_KEY=sk-ant-...

# Gmail API
GMAIL_CREDENTIALS_PATH=./config/credentials.json
GMAIL_TOKEN_PATH=./config/token.json
```

```
# Polling interval (seconds)
POLL_INTERVAL=30
```

6.5.2 תצורת השופט

vne.eerefer

```
# Referee Configuration
REFEREE_EMAIL=MyGroup.referee@gmail.com
REFEREE_NAME=MyGroup Referee
LEAGUE_MANAGER_EMAIL=bitalevi100@gmail.com
LOG_SERVER_EMAIL=beit.halevi.700@gmail.com

# Course Materials
MATERIALS_PATH=./data/course_materials/

# LLM Configuration
LLM_PROVIDER=anthropic
LLM_MODEL=claude-sonnet-4-20250514
LLM_API_KEY=sk-ant-...

# Gmail API
GMAIL_CREDENTIALS_PATH=./config/credentials.json
GMAIL_TOKEN_PATH=./config/token.json

# Polling interval (seconds)
POLL_INTERVAL=30
```

6.5.3 קובץ הגדרות מפורט לשופט (config.json)

קובץ ההגדרות המלא כולל את כל הפרמטרים הניתנים לכוונון:

noitarugifnoC lluF eerefeR — nosj.gifnoc

```
{
  "version": "1.0.0",
  "protocol_version": "league.v2",

  "gmail": {
    "poll_interval_seconds": 30,
    "max_emails_per_minute": 10,
    "batch_size": 10
  },
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
  "database": {
    "host": "localhost",
    "port": 5432,
    "name": "q21_referee",
    "pool_size": 5
  },

  "timeouts": {
    "warmup_seconds": 300,
    "warmup_retries": 1,
    "questions_seconds": 600,
    "guess_seconds": 600,
    "default_seconds": 900
  },

  "q21": {
    "max_questions": 20,
    "demo_ai_mode": false,
    "scoring_weights": {
      "opening_sentence": 0.50,
      "sentence_justification": 0.20,
      "associative_word": 0.20,
      "word_justification": 0.10
    },
    "league_score_thresholds": {
      "excellent": 85,
      "good": 70,
      "average": 50
    },
    "feedback_word_count": {
      "min": 30,
      "max": 50
    }
  },

  "logging": {
    "level": "INFO",
    "sensitive_patterns": ["password", "token", "secret"]
  }
}
```

טבלה 65: פרמטרים חשובים ב-config.json

פרמטר	ערך ברירת מחדל	תיאור
timeouts.warmup_seconds	300	זמן תפוגה לשלב חימום
timeouts.questions_seconds	600	זמן תפוגה לשלב שאלות
timeouts.guess_seconds	300	זמן תפוגה לשלב ניחוש
q21.max_questions	20	מספר שאלות מקסימלי
q21.demo_ai_mode	false	מצב הדגמה (תשובות מוכנות)
q21.feedback_word_count.min	30	מינימום מילים בהסבר
q21.feedback_word_count.max	50	מקסימום מילים בהסבר

6.6 בדיקות

6.6.1 אסטרטגיות בדיקה

1. **בדיקות יחידה** — בדיקת פונקציות בודדות
2. **בדיקות אינטגרציה** — בדיקת תקשורת עם Gmail API
3. **בדיקות קצה-לקצה** — משחק מלא בין סוכנים
4. **בדיקות מול עצמכם** — שחקו נגד השופט שלכם

6.6.2 רשימת בדיקות חובה

טבלה 66: רשימת בדיקות חובה

בדיקה	תיאור
רישום	הסוכן נרשם בהצלחה ומקבל מזהה
תגובה להזמנה	השחקן מגיב להזמנת משחק בזמן
יצירת שאלות	השחקן מייצר 20 שאלות תקינות
ניחוש סופי	השחקן שולח ניחוש עם כל הרכיבים
חישוב ציון	השופט מחזיר ציון תקין עם פירוט
לשרת הלוג CC	כל הודעה כוללת CC
עמידה במועדים	תגובות בתוך חלונות הזמן
GateKeeper	מגבלות קצב ומכסות מיושמות
טיפול בשידורים	תגובה לשידורים שדורשים תגובה
בקשת הארכה	יכולת לבקש הארכה לפני דדליין
מעקב הודעות	טבלת מעקב עם סטטוסים

כל הזכויות שמורות - © Dr. Segal Yoram

6.6.3 בדיקות ייעודיות לשופט

סוכן השופט דורש בדיקות נוספות מעבר לבדיקות הכלליות:

טבלה 67: בדיקות ייעודיות לשופט

בדיקה	תיאור
מכונת מצבים	מעבר תקין בין 12 מצבי המשחק
ניהול זמני תפוגה	התראות ופעולות בתום זמן
חישוב ציונים	חישוב נכון של 4 רכיבי הציון
המרה לציון ליגה	המרה נכונה מציון פרטי (0–100) לליגה (0–3)
משוב מילולי	אורך הסבר 30–50 מילים לכל רכיב
דיווח למנהל	שליחת MATCH_RESULT_REPORT תקין
התאוששות מקריסה	המשך משחק ממצב שנשמר
Warmup עם ניסיון חוזר	טיפול ב-timeout עם ניסיון אחד נוסף

Example Referee Test Structure

```
# tests/referee/
# ├── unit/
# │   ├── test_score_calculator.py    # Scoring logic
# │   ├── test_state_machine.py       # State transitions
# │   └── test_timeout_manager.py     # Timeout handling
# ├── integration/
# │   ├── test_gmail_handler.py       # Email processing
# │   └── test_database_ops.py        # DB operations
# └── e2e/
#     └── test_full_game.py           # Complete game flow

import pytest

class TestScoreCalculator:
    """Unit tests for referee scoring."""

    def test_league_score_conversion_excellent(self):
        """Score 85+ should give 3 league points."""
        calc = ScoreCalculator()
        assert calc._convert_to_league_score(85) == 3
        assert calc._convert_to_league_score(100) == 3

    def test_league_score_conversion_boundaries(self):
        """Test boundary conditions."""
        calc = ScoreCalculator()
        assert calc._convert_to_league_score(84.99) == 2
```

```
        assert calc._convert_to_league_score(69.99) == 1
        assert calc._convert_to_league_score(49.99) == 0

    def test_feedback_word_count(self):
        """Feedback must be 30-50 words."""
        calc = ScoreCalculator()
        feedback = calc._generate_feedback(original, guess, components)
        word_count = len(feedback["sentence"].split())
        assert 30 <= word_count <= 50
```

6.7 מימוש טבלת מעקב הודעות

הסוכן חייב לעקוב אחר כל הודעה שנשלחה ושהתקבלה. הטבלה מאפשרת זיהוי תגובות חסרות ומניעת שליחה כפולה.

6.7.1 מבנה הטבלה

טבלה 68: מבנה טבלת message_logs

שדה	סוג	תיאור
id	INTEGER (PK)	מזהה אוטומטי
gmail_id	VARCHAR(100)	מזהה ייחודי של ההודעה (Gmail ID)
thread_id	VARCHAR(100)	מזהה השרשור
direction	VARCHAR(10)	כיוון: "IN" (קבלה) או "OUT" (שליחה)
message_type	VARCHAR(100)	סוג ההודעה
transaction_id	VARCHAR(100)	מזהה טרנזקציה
sender_email	VARCHAR(255)	כתובת השולח
recipient_email	VARCHAR(255)	כתובת הנמען
subject	TEXT	נושא ההודעה
payload	JSON	תוכן ההודעה
game_id	VARCHAR(100)	מזהה משחק
league_id	VARCHAR(100)	מזהה ליגה
round_id	VARCHAR(100)	מזהה סיבוב
processed	BOOLEAN	האם עובד
error_message	TEXT	הודעת שגיאה (אם יש)
created_at	TIMESTAMP	זמן יצירה
processed_at	TIMESTAMP	זמן עיבוד

כל הזכויות שמורות - © Dr. Segal Yoram

קורלציית הודעות

קישור בקשה-תגובה מתבצע בטבלה נפרדת message_correlations עם השדות: quest_type, request_id, response_type, response_id, correlated_at.

6.7.2 מימוש מעקב

מחלקת מעקב הודעות

```
from datetime import datetime, timedelta
from enum import Enum
from dataclasses import dataclass
from typing import Optional

class MessageStatus(Enum):
    OPEN = "OPEN"       # Waiting for response
    CLOSED = "CLOSED"   # Response received
    REJECTED = "REJECTED"  # Deadline passed

@dataclass
class TrackedMessage:
    message_id: str
    correlation_id: str
    message_type: str
    deadline: Optional[datetime]
    status: MessageStatus = MessageStatus.OPEN

class MessageTracker:
    def __init__(self):
        self.messages: dict[str, TrackedMessage] = {}

    def track_sent(self, msg: dict, deadline_seconds: int = None):
        """Track outgoing message with optional deadline."""
        deadline = None
        if deadline_seconds:
            deadline = datetime.now() + timedelta(seconds=
   deadline_seconds)
        tracked = TrackedMessage(
            message_id=msg["message_id"],
            correlation_id=msg.get("correlation_id"),
            message_type=msg["message_type"],
            deadline=deadline
        )
        self.messages[msg["correlation_id"]] = tracked

    def close_by_correlation(self, correlation_id: str):
        """Mark message as closed when response received."""
        if correlation_id in self.messages:
            self.messages[correlation_id].status = MessageStatus.CLOSED
```


© Dr. Segal Yoram - כל הזכויות שמורות

6.8 מימוש מטפל שידורים

השרת שולח 11 סוגי שידורים שונים. חלקם דורשים תגובה ויש להם דדליין.

6.8.1 זיהוי שידורים

זיהוי סוג ההודעה

```
# Broadcast types that require response
RESPONSE_REQUIRED = {
    "BROADCAST_CONNECTIVITY_TEST": 60,      # 60 seconds
    "BROADCAST_KEEP_ALIVE": 300,            # 5 minutes
    "BROADCAST_CRITICAL_SYSTEM": 120,       # 2 minutes
    "BROADCAST_CRITICAL_SECURITY": 120,     # 2 minutes
}

# Broadcast types - no response needed
NO_RESPONSE_NEEDED = {
    "BROADCAST_SEASON_START",
    "BROADCAST_ROUND_START",
    "BROADCAST_ROUND_END",
    "BROADCAST_SEASON_END",
    "BROADCAST_ASSIGNMENT_TABLE",
    "BROADCAST_STANDINGS_UPDATE",
    "BROADCAST_MAINTENANCE_NOTICE",
}

def get_response_deadline(message_type: str) -> int | None:
    """Get deadline in seconds, or None if no response needed."""
    return RESPONSE_REQUIRED.get(message_type)
```

6.8.2 טיפול בשידורים

מטפל שידורים

```
class BroadcastHandler:
    def __init__(self, email_client, tracker: MessageTracker):
        self.email_client = email_client
        self.tracker = tracker

    async def handle_broadcast(self, msg: dict):
        """Process incoming broadcast message."""
        msg_type = msg["message_type"]
        deadline_seconds = get_response_deadline(msg_type)
```

```
        if deadline_seconds:
            # Response required - track and respond
            self.tracker.track_sent(msg, deadline_seconds)
            await self._send_broadcast_response(msg, deadline_seconds)
        else:
            # Informational only - log and process
            self._process_informational(msg)

    async def _send_broadcast_response(self, msg: dict, deadline: int):
        """Send acknowledgment response to broadcast."""
        response = {
            "message_type": f"RESPONSE_{msg['message_type']}",
            "correlation_id": msg["message_id"],
            "payload": {"status": "ACKNOWLEDGED"}
        }
        await self.email_client.send(response)
```

6.9 מימוש בקשת הארכה

כאשר הסוכן צופה שלא יספיק לעמוד בדדליין, הוא יכול לבקש הארכה.

6.9.1 מתי לבקש הארכה

כלל הזהב

בקשת הארכה חייבת להישלח **לפני** שפג תוקף הדדליין. בקשה מאוחרת תידחה!

6.9.2 מימוש מבקש הארכה

מנהל הארכות

```
class ExtensionManager:
    def __init__(self, email_client, tracker: MessageTracker):
        self.email_client = email_client
        self.tracker = tracker
        self.pending_extensions: dict[str, bool] = {}

    async def request_if_needed(
        self,
        original_msg: dict,
        deadline: datetime,
        estimated_processing: float
    ) -> bool:
        """Request extension if we won't make deadline."""
```


כל הזכויות שמורות - © Dr. Segal Yoram

```
        time_remaining = (deadline - datetime.now()).total_seconds()
        buffer = 30  # Safety margin

        if time_remaining < estimated_processing + buffer:
            return await self._send_extension_request(original_msg)
        return True  # No extension needed

    async def _send_extension_request(self, original_msg: dict) -> bool:
        """Send extension request and wait for response."""
        request = {
            "message_type": "EXTENSION_REQUEST",
            "payload": {
                "original_message_id": original_msg["message_id"],
                "original_message_type": original_msg["message_type"],
                "reason": "Processing requires additional time",
                "requested_extension_seconds": 120
            }
        }
        correlation_id = request.get("correlation_id")
        self.pending_extensions[correlation_id] = None

        await self.email_client.send(request)
        # Wait for RESPONSE_EXTENSION...
        return await self._wait_for_response(correlation_id)

    def handle_extension_response(self, response: dict):
        """Process extension response from server."""
        correlation_id = response.get("correlation_id")
        if correlation_id in self.pending_extensions:
            status = response["payload"]["status"]
            self.pending_extensions[correlation_id] = (status == "
APPROVED")
```

6.10 הרצת הליגה

6.10.1 שלבי ההפעלה

1. הגדירו את קבצי התצורה
2. הפעילו את סוכן השופט: `python main.py referee`
3. הפעילו את סוכן השחקן: `python main.py player`
4. הסוכנים יירשמו אוטומטית ויחכו להודעות
5. בצעו בדיקות מול עצמכם לפני הליגה הרשמית

6.10.2 טיפים להצלחה

טיפ למימוש

טיפים להצלחה בליגה:

- **לוגים** — תעדו כל הודעה נכנסת ויוצאת
- **מועדים** — הוסיפו מרווח ביטחון (אל תחכו לשנייה האחרונה)
- **טיפול בשגיאות** — טפלו בכל חריגה, כולל שגיאת 429 (Rate Limit)
- **מצב יציב** — שמרו מצב למקרה של הפעלה מחדש
- **תרגול** — שחקו מול עצמכם שוב ושוב
- **מגבלות API** — הכירו את מגבלות Gmail API (ראו נספח ה')

6.11 סיכום

פרק זה סיפק הנחיות מעשיות למימוש:

- הגדרת סביבת פיתוח עם Python 3.10+ ו-Gmail API
- מבנה פרויקט מומלץ עם הפרדה לשחקן ושופט
- מימוש סוכן שחקן עם יצירת שאלות ואסטרטגיית ניחוש
- מימוש סוכן שופט עם בחירת פסקה וחישוב ציונים
- קבצי תצורה לשני הסוכנים
- אסטרטגיות בדיקה ורשימת בדיקות חובה

בהצלחה בפרויקט הגמר!

כל הזכויות שמורות - Dr. Segal Yoram ©